



Original software publication

eeglib: A Python module for EEG feature extraction

Luis Cabañero-Gomez^{*}, Ramon Hervas, Ivan Gonzalez, Luis Rodriguez-Benitez

Department of Information Systems and Technologies, University of Castilla-La Mancha, Ciudad Real 13071, Spain



ARTICLE INFO

Article history:

Received 30 April 2021

Received in revised form 2 June 2021

Accepted 11 June 2021

Keywords:

EEG

Biosignals

Signal analysis

Python

ABSTRACT

Electroencephalography (EEG) signals analysis is non-trivial, thus tools for helping in this task are crucial. One typical step in many studies is feature extraction, however, there are not many tools focused on that aspect. In this paper, eeglib: a Python library for EEG feature extraction is presented. It includes the most popular algorithms when working with EEG and can be easily combined with popular Python libraries. This paper also presents a simple workflow for creating features dataset which allows a high degree of customization and that is suitable for both experts and newcomers.

© 2021 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Code metadata

Current code version

0.4.0

Permanent link to code/repository used of this code version

<https://github.com/ElsevierSoftwareX/SOFTX-D-21-00087>

Code Ocean compute capsule

Legal Code License

Code versioning system used

MIT

Software code languages, tools, and services used

git

Compilation requirements, operating environments & dependencies

Python

If available Link to developer documentation/manual

See <https://github.com/Xiul109/eeglib/blob/master/requeriments.txt>

Support email for questions

<https://eeglib.readthedocs.io/en/latest/>luis.cabanero@uclm.es

1. Motivation and significance

Electroencephalography (EEG) is a technique which allows to obtain inputs of the electric potential produced by the brain activity. This is usually achieved by placing electrodes over the scalp, so it is not an invasive technique, although there are some versions that require surgery. EEG analysis can be used in many fields like education, neuromarketing, and medicine, where it is used for many applications such as epilepsy diagnosis [1] or as a supplementary tool for lots of brain disorders [2]. Signal analysis is often complex and nontrivial; however, this problem is even larger with EEG signals. This happens not only because this type of signals usually contains more than one channel but also because they are non-linear, non-stationary and chaotic [3]. For these reasons there is a specific approach when analysing this kind of signals which have led to the creation of software specifically oriented to deal with them. Probably the most used

general tool for EEG signal analysis is EEGLab [4], an open source toolbox for MatLab, while the most used Python tool would be MNE-Python [5], which is also open source. Both of these tools are exhaustive and really powerful, nevertheless they are general tools, so they are not so suitable for specific purposes. For example, EEGsynth is a powerful tool to create brain-computer interfaces that connect open-hardware physiological recorders with output devices like lights or synthesizers and it is often used for artistic purposes. None of these tools is feature-extraction-oriented, which makes them non suitable for certain types of analysis, for example machine learning based ones. There is also a barrier for the user that approaches to these libraries that is the requirement of a high degree of knowledge about neuroscience and EEG.

In response to these problems, we present eeglib, an open source Python library which is a powerful feature extraction tool oriented towards EEG signals and based on sliding windows. The design of eeglib is oriented towards compatibility with the most used machine learning and data analysis libraries for Python, so its output can be an input for them, which simplifies the workflow. Despite being a useful tool for EEG experts, it also can be manageable for newcomers to the field. There are not many

^{*} Corresponding author.

E-mail addresses: luis.cabanero@uclm.es (Luis Cabañero-Gomez), ramon.hlucas@uclm.es (Ramon Hervas), ivan.gdiaz@uclm.es (Ivan Gonzalez), luis.rodriguez@uclm.es (Luis Rodriguez-Benitez).

other EEG feature extraction libraries, being PyEEG [6] one, but most of them just offer a set of useful functions while eeglib offers tools that help the user to easily create a whole dataset and auto-parametrizes the algorithms, which simplifies the analysis if the user is not an expert.

2. Software description

2.1. Software architecture

The eeglib module is composed of 6 different submodules whose structure is depicted in Fig. 1. The *features* submodule includes a set of functions to compute each of the features included in the library. The *eeg* submodule defines a data structure designed to store fragments of EEG signals data and apply different types of processing included in *features* to it. The *helpers* submodule contains Helper classes that allow data loading and iteration over the data using sliding windows, which are instances of the data structures defined in *eeg* submodule. The *wrapper* submodule wraps around a Helper to extract a set of features defined by the user for each window defined in it. For last, the *preprocessing* submodule includes functions to pre-process the signal and *auxFunctions* includes some auxiliary functions not aimed to the final user.

2.2. Software functionalities

Below the main functionalities of eeglib are described. This library is not just a collection of functions for computing an extensive set of features from EEG signals, it also contains many tools to make the process easier for the user.

2.2.1. Data loading and preprocessing

Data can be loaded from three different sources: Comma Separated Values (CSV) files, European Data Format (EDF) and NumPy [7] arrays. CSV files are a usual format for many kinds of dataset, including EEG signal data, however it does not carry information about the sample rate of the data, so it is up to the user configure it. EDF are a standard for biosignals which includes data about the signal like the sample rate and the physical units. There are lots of formats besides these two, but they are the most used, however if the user needs to load any other format, it can be done by loading it with another tool, converting it into NumPy arrays and feeding it to the module.

As stated before, eeglib is a sliding-window based tool, which means it applies the feature extraction over segments of the data across the whole signal. These segments can or not overlap one with another if required for the analysis. At data loading, the user can define how windows will be disposed by specifying two parameters: the window size and the window displacement.

Despite the main capability of the library is not preprocessing EEG signals, which is a very large field, it includes some basic and yet useful tools. These tools are bandpass filter, Independent Component Analysis (ICA) [8] and z-scores normalization [9]. In eeglib, preprocessing is applied at data loading step because it is applied to the whole signal, contrary to features, that are extracted from windows.

2.2.2. Feature extraction

Every feature currently implemented in eeglib is displayed in Table 1 grouped in two main categories and four subcategories. The first categorization level corresponds to the number of channels over which a feature is applied: one single channel or two channels. Most of the features are applied over a single channel and they are divided into four more categories:

- Transformations. They change the domain of the input signal, usually from time domain to frequency domain.
- Frequency domain. The features in this category give information obtained from the frequency domain.
- Time domain. The features in this category give information obtained from the time domain.
- Complexity. There are lots of algorithms that evaluate the complexity of a signal, which is a wide concept that groups the randomness of a signal, the fractal dimension and the self-affinity.

Besides the features specified in the table, it is also possible to include custom algorithms when performing a feature extraction process.

2.2.3. Dataset generation

One of the main objectives of eeglib is the generation of dataset compatible with most of the data analysis libraries for Python. That capability belongs to the *wrapper* submodule, which provides a tool to generate a pandas [23] DataFrame containing the features desired by the user. This type of output is useful by itself, however for many use cases it can be improved with one of the utilities eeglib provides: dataset labelling. There are two kinds of labels: general labels and segments related labels. The general label refers to a label which is the same to the whole generated dataset. On the other hand, the segment labels are set in specific temporal fragments and they can be different between them. General labels are useful to classify full trials, while segment labels can be used to classify specific events or states inside a trial, and both can be combined if needed.

2.3. Testing

The library has been properly tested and validated to ensure it does not contain critical errors. This testing is done in two ambits: the general usage and the algorithms output. The general usage testing consists of covering almost every line of code by setting different combinations of parameters than the user can pass to functions and methods. With this type of testing, we want to check that the library behaves as intended and according to the documentation. This kind of testing is not enough to ensure that the implementation of the algorithms is correct, so it is necessary to perform other type of testing. The algorithms output testing consists of comparing implementation output using as input signals whose results are known beforehand. For most of the algorithms, these signals are random generated as a coloured noise: white noise, pink noise, or Brownian noise, but some of them use pre-generated signals stored in a file.

3. Illustrative examples

The intended manner to use the library consists of four main steps which are depicted in Fig. 2. The first (1) step is data loading, and it is performed through a *Helper* object. Currently there are 3 types of these objects, one for loading CSV files, one for EDF files and one that accepts arrays instead of a file name, which gives flexibility for other file formats. This step also includes the configuration of some settings to the data like the preprocessing to be applied, the sliding window configuration or specific names for each channel. For this case a CSV file is loaded with a sample rate of 256, without applying any other configuration. The next step (2) is the instantiation of a *Wrapper* object around the previously created *Helper*. In this step the user can define the format of the output data, as well as adding labels to the output data which are useful for machine learning applications. The third step (3) consist of asking the *Wrapper* for the features that are going to be extracted, parametrizing the functions as needed. For this

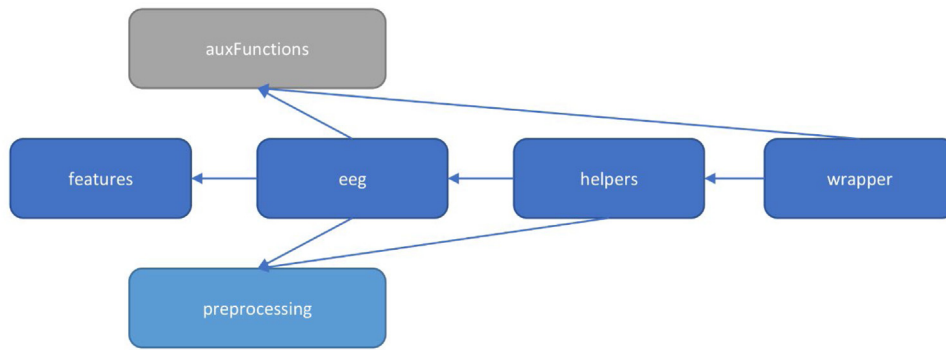


Fig. 1. Dependency diagram of the different eeglib submodules. The darker blue cells represent the main processing pipeline. The *preprocessing* submodule is in light blue because its utility is secondary to the user and the *auxFunctions* module is in grey because it is not relevant to the final user. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Table 1

Summary of the features that can be extracted through eeglib by default.

Single-channel	Transformations	Fast Fourier Transform [10] Discrete Wavelet Transform [11] Power Spectral Density [12]
	Frequency domain	Band Power [13]
	Time domain	Hjorth Parameters [14]
	Complexity	Detrended Fluctuation Analysis [15] Sample Entropy [16] Lempel–Ziv Complexity [17] Petrosian Fractal Dimension [18] Higuchi Fractal Dimension [19]
	Dual-channel	Pearson Cross-Correlation Coefficient [20] Dynamic Time Warping [21] Synchronization Likelihood [22]

example, the included features use the default parametrization, but two of them are applied only to specific channels. For last (4), the features are extracted creating a dataset suitable to further analysis. This example shows a basic procedure to follow, but there are more tools in the library that allows the user a higher degree of customization.

In Fig. 3 there is an example based in a real-world case. The data comes from an EDF file containing an EEG signal with 14 channels recorded at 128 Hz with a duration of 11 s. It is known that the recorded person was performing an action the first 5 s and then changed to another in the rest of the recording and the goal is to create a classifier capable to distinguish the two actions. This time, in the data loading step (1) some preprocessing is performed, which consist of a z-scores normalization to centre the data with a highpass filter at 1 Hz and a lowpass filter at 50 Hz to erase a big part of the noise from the original signal. As the EDF files include sample rate data, it is not necessary to specify it. Also, only four specific channels are selected because the rest are not relevant in this example.

By default, the window size is equal to the sample rate, which correspond to one second of the recording, but in this case, it has been changed (2) to two times the sample rate or two seconds. It can also be specified at data loading step, however in this case it has been done after it to show how to retrieve the sample rate from the helper.

After it (3), a minor setting has been changed to change the way the output of the algorithms is returned. By default, it is returned as an array with a value for each channel, but if it is changed to “dict” mode, the output will be a dictionary whose keys correspond to the names of the channels and its values to the output of the features that will be specified. This is useful to make the data output more readable to humans.

The Wrapper instantiation (4) is done in a similar way than before, but there are two differences here. The first of them is

that the helper object takes an index, which is a way to define the limits of the sliding window processing. It is used as *helper[start:stop:step]*, where start is where the first window will begin, stop is where the last window will end and step if the offset between windows. If the value is written within quotation marks it is interpreted as time, but if it is a numeric value it is interpreted as the number of samples. In the example, the sliding window is settled to start at the beginning of the signal and stop at its end, which are the default values, but the step is modified to slide half second each time. The other difference with the previous example regarding the Wrapper is the inclusion of a segmentation according to the data description in which the first five seconds receive the label 0 and the last six receive the label 1.

For last (5), band power is specified to be extracted and the features in each window are computed. Band power is the estimation of the energy associated to various frequency bands and by default is applied to theta (1–4 Hz), delta (4–7 Hz), alpha (8–12 Hz) and beta (12–30 Hz) bands, but other can be specified by the user. The parameter set to true when adding the feature to the wrapper is used to hide the parameters used in the output, which is only advisable to do when the same feature is added just one time.

In Fig. 4, there is an example about how to use the data obtained after the whole process to train a support vector machine classifier from the library scikit-learn [24]. As displayed in the picture, the segmentation is in “segment_label” column of the DataFrame, which can be easily extracted and used to feed the classifier.

4. Impact

Through published papers, eeglib has proved its efficacy and utility. The first appearance of eeglib in a peer-reviewed publication was in [25], in which it is explained how most of the

```

import eeglib

helper = eeglib.helpers.CSVHelper("newEEG.csv", sampleRate=256) # (1)

wrapper = eeglib.wrapper.Wrapper(helper) # (2)

wrapper.addFeature.PFD() # (3)
wrapper.addFeature.HFD([0,2,8,10]) # (3)
wrapper.addFeature.DTW((0,1)) # (3)

data = wrapper.getAllFeatures() # (4)

```

Fig. 2. Code snippet showing the general procedure when using eeglib.

```

import eeglib
# Data loading (1)
helper = eeglib.helpers.EDFHelper("EEG.edf",
                                   normalize = True, highpass = 1, lowpass = 50,
                                   selectedSignals=["O1", "O2", "F3", "F4"])

# Setting the window size (2)
helper.prepareEEG(helper.sampleRate*2)

# This line changes the output to show the channel names (3)
helper.eeg.outputMode = "dict"

# Wrapper creation with segmentation (4)
wrap = eeglib.wrapper.Wrapper(helper[:, "0.5"], segmentation=[(("0", "5"), 0),
                                                                (("5", "11"), 1)])

# For this example only standard bands power is extracted (5)
wrap.addFeature.bandPower(hideArgs = True)
data = wrap.getAllFeatures()

```

Fig. 3. Code snippet showing an applied example of eeglib.

```

# Using the data in a classifier
from sklearn.svm import SVC

X = data.drop(columns = ["segment_label"])
y = data["segment_label"]

clf = SVC()
clf.fit(X, y)

```

Fig. 4. Combining eeglib output with other libraries.

features that it is able to extract were chosen. Also, that article shows the efficacy of eeglib using EEG data from real experiments and how some of the implemented algorithms are tested by using synthetic signals created as coloured noises.

The library has also been used in two highly related papers [26, 27] in which the cognitive load of different usual tasks performed when interacting with mobile phones are analysed. Despite both papers having the same purpose, each of them used a data from different experiments. Thanks to eeglib it was possible to obtain an estimation of the cognitive load from the EEG data.

The current importance and relevance of machine learning in EEG and the capabilities of eeglib to generate datasets with few lines makes this tool suitable for future research. It can be applicable to Brain-Computer Interfaces research or to cognitive states research. It can even be used for deep learning, because, even if deep learning can deal with the raw data without the need of extract features, eeglib can provide the data segmented in windows of a consistent length with few lines, which is the format usually utilized by deep learning algorithms. The module can also be integrated in other projects, even commercial ones.

5. Conclusions

In the current version, eeglib is a totally functional and useful library for EEG analysis and feature extraction, which has been proved to be useful in several scientific publications and that has been tested in deep. It is capable to extract some of the most relevant features from EEG signals and, if a feature is not included, it can be easily integrated by the user. It presents an easy workflow which simplifies the coding phase when analysing data and that is simple to modify. It is affordable for newcomers to EEG and neurosciences fields, but also useful to experts.

Despite being a powerful tool, it can be further improved. One of the main improvements that can be done is the addition of more features to the library. Even if CSV and EDF files are some of the more popular ones used for EEG, they are not the only ones and eeglib could improve by including to it compatibility with other file formats. Another limitation related with the input signal is real-time access to devices, which is something allowed by the architecture of eeglib, but it has not been implemented yet. Also, even if the library is designed to be able to process the data with few lines of code, this aspect can be improved in lots of ways, for example, making it able to process at once more than one file with only one statement.

Funding

The development of eeglib was funded by the MINISTERIO DE CIENCIA, INNOVACIÓN Y UNIVERSIDADES, grant number RTI2018-098780-B-I00 (national research project) and the PI001097 predoctoral contract by JUNTA DE COMUNIDADES DE CASTILLA-LA MANCHA.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] Shoeibi A, et al. A comprehensive comparison of handcrafted features and convolutional autoencoders for epileptic seizures detection in EEG signals. *Expert Syst Appl* 2021;163:113788. <http://dx.doi.org/10.1016/j.eswa.2020.113788>.
- [2] Tzamourta KD, et al. Machine learning algorithms and statistical approaches for alzheimer's disease analysis based on resting-state EEG recordings: A systematic review. *Int J Neural Syst* 2021;31(5):2130002. <http://dx.doi.org/10.1142/S0129065721300023>.
- [3] Pijn JP, Van Neerven J, Noest A, Lopes da Silva FH. Chaos or noise in EEG signals; dependence on state and brain site. *Electroencephalogr Clin Neurophysiol* 1991;79(5):371–81. [http://dx.doi.org/10.1016/0013-4694\(91\)90202-F](http://dx.doi.org/10.1016/0013-4694(91)90202-F).
- [4] Delorme A, Makeig S. EEGLAB: an open source toolbox for analysis of single-trial EEG dynamics including independent component analysis. *J Neurosci Methods* 2004;134(1):9–21. <http://dx.doi.org/10.1016/j.jneumeth.2003.10.009>.
- [5] Gramfort A, et al. MEG and EEG data analysis with MNE-Python. *Front Neurosci* 2013;7. <http://dx.doi.org/10.3389/fnins.2013.00267>.
- [6] Bao FS, Liu X, Zhang C. PyEEG: an open source Python module for EEG/MEG feature extraction. *Comput Intell Neurosci* 2011;2011:406391. <http://dx.doi.org/10.1155/2011/406391>.
- [7] Harris CR, et al. Array programming with NumPy. *Nature* 2020;585(7825):7825. <http://dx.doi.org/10.1038/s41586-020-2649-2>.
- [8] Comon P. Independent component analysis, A new concept? *Signal Process* 1994;36(3):287–314. [http://dx.doi.org/10.1016/0165-1684\(94\)90029-9](http://dx.doi.org/10.1016/0165-1684(94)90029-9).
- [9] Tarantola S. Z-scores. In: Michalos AC, editor. *Encyclopedia of quality of life and well-being research*. Dordrecht: Springer Netherlands; 2014, p. 7316. http://dx.doi.org/10.1007/978-94-007-0753-5_3307.
- [10] Cooley JW, Tukey JW. An algorithm for the machine calculation of complex Fourier series. *Math Comp* 1965;19(90):297–301. <http://dx.doi.org/10.2307/2003354>.
- [11] Mallat SG. A theory for multiresolution signal decomposition: the wavelet representation. *IEEE Trans Pattern Anal Mach Intell* 1989;11(7):674–93. <http://dx.doi.org/10.1109/34.192463>.
- [12] Youngworth RN, Gallagher BB, Stamper BL. An overview of power spectral density (PSD) calculations. In: *Optical manufacturing and testing VI*. 2005, p. 5869, 58690U. <http://dx.doi.org/10.1117/12.618478>.
- [13] Bandpower of an EEG signal. 2021. <https://raphaelvallat.com/bandpower.html>. [Accessed 11 March 2021].
- [14] Hjorth B. EEG analysis based on time domain properties. *Electroencephalogr Clin Neurophysiol* 1970;29(3):306–10. [http://dx.doi.org/10.1016/0013-4694\(70\)90143-4](http://dx.doi.org/10.1016/0013-4694(70)90143-4).
- [15] Peng CK, Buldyrev SV, Havlin S, Simons M, Stanley HE, Goldberger AL. Mosaic organization of DNA nucleotides. *Phys Rev E* 1994;49(2):1685–9.
- [16] Richman JS, Moorman JR. Physiological time-series analysis using approximate entropy and sample entropy. *Am J Physiol-Heart Circ Physiol* 2000;278(6):H2039–49. <http://dx.doi.org/10.1152/ajpheart.2000.278.6.H2039>.
- [17] Hu J, Gao J, Principe JC. Analysis of biomedical signals by the Lempel–Ziv complexity: the effect of finite data size. *IEEE Trans Biomed Eng* 2006;53(12):2606–9. <http://dx.doi.org/10.1109/TBME.2006.883825>.
- [18] Petrosian A. Kolmogorov complexity of finite sequences and recognition of different preictal EEG patterns. In: *Proceedings eighth IEEE symposium on computer-based medical systems*. 1995, p. 212–7. <http://dx.doi.org/10.1109/CBMS.1995.465426>.
- [19] Higuchi T. Approach to an irregular time series on the basis of the fractal theory. *Physica D* 1988;31(2):277–83. [http://dx.doi.org/10.1016/0167-2789\(88\)90081-4](http://dx.doi.org/10.1016/0167-2789(88)90081-4).
- [20] Benesty J, Chen J, Huang Y, Cohen I. Pearson correlation coefficient. In: Cohen I, Huang Y, Chen J, Benesty J, editors. *Noise reduction in speech processing*. Berlin, Heidelberg: Springer; 2009, p. 1–4. http://dx.doi.org/10.1007/978-3-642-00296-0_5.
- [21] Müller M, editor. *Dynamic time warping*. In: *Information retrieval for music and motion*. Berlin, Heidelberg: Springer Berlin Heidelberg; 2007, p. 69–84. http://dx.doi.org/10.1007/978-3-540-74048-3_4.
- [22] Stam CJ, van Dijk BW. Synchronization likelihood: an unbiased measure of generalized synchronization in multivariate data sets. *Physica D* 2002;163(3):236–51. [http://dx.doi.org/10.1016/S0167-2789\(01\)00386-4](http://dx.doi.org/10.1016/S0167-2789(01)00386-4).
- [23] Reback Jeff, et al. *Pandas-Dev/Pandas: Pandas 1.0.3*. Zenodo; 2020, <http://dx.doi.org/10.5281/zenodo.3715232>.
- [24] Pedregosa F, et al. *Scikit-learn: Machine learning in python*. *J Mach Learn Res* 2011;12(85):2825–30.
- [25] Cabañero L, Hervás R, Bravo J, Rodríguez-Benitez L, Nugent C. eeglib: computational analysis of cognitive performance during the use of video games. *J Ambient Intell Human Comput* 2019. <http://dx.doi.org/10.1007/s12652-019-01592-9>.
- [26] Cabañero L, Hervás R, González I, Fontecha J, Mondéjar T, Bravo J. Analysis of cognitive load using EEG when interacting with mobile devices. *Proceedings 2019*;31(1):70. <http://dx.doi.org/10.3390/proceedings2019031070>.
- [27] Cabañero L, Hervás R, González I, Fontecha J, Mondéjar T, Bravo J. Characterisation of mobile-device tasks by their associated cognitive load through EEG data processing. *Future Gener Comput Syst* 2020;113:380–90. <http://dx.doi.org/10.1016/j.future.2020.07.013>.